Python tooling: uv, pyproject, venvs, ruff, VS Code

Week 1 · `w01-py-tooling` · built 2026-10-15 · docs verified 2026-10-11 to 2026-10-15 (uv 0.12.24, ruff 0.17.0, pytest 9.1.1, CPython 3.13.16)

Summary

Python has no `javac` and no single Maven, so the toolchain does more of the safety work. **uv** installs and picks Python versions, resolves and locks dependencies, and runs everything inside the project's own virtual environment. **ruff** lints and formats, and stands in for most of what the Java compiler and Checkstyle catch. **pytest** runs tests found by naming convention. Every course task is a uv project that has to be ruff-clean with its tests passing, and every LLM service you ship later needs the same reproducible environment in CI and in Docker.

Key concepts

- **Interpreter selection**: the shell runs the first file on `PATH` whose name matches the command exactly (`python3` ≠ `python3.13`). Packages live inside one interpreter's environment, so the interpreter decides which imports work.
- **`pyproject.toml`**: project metadata, dependency *ranges* and tool config (ruff, pytest). It's the `pom.xml`.
- **`.python-version`**: the project's interpreter pin; `uv init --python 3.13` writes it. Without `--python`, uv uses the first interpreter it finds and floors `requires-python` at that version.
- **`uv.lock`**: the exact resolved versions and hashes of every direct and transitive dependency. Always committed, never edited by hand.
- **Resolution**: without a lock, every range resolves to the **newest** version allowed. With a lock, `uv sync` and `uv run` install exactly the locked versions until you upgrade on purpose.
- **`.venv`**: a folder holding an interpreter shim plus `site-packages`. It's the per-project classpath. Never committed.
- **`uv run`**: lock and sync if needed, then run inside `.venv`. That makes `sys.path` this project's classpath.
- **Dependency groups**: `uv add --dev` puts tools in `[dependency-groups] dev`, the equivalent of `<scope>test</scope>`.
- **ruff check / ruff format**: lint (report, and fix with `--fix`) vs format (whitespace and wrapping). `select` *replaces* the default rules; `extend-select` adds to them.
- **Safe vs unsafe fixes**: `--fix` applies only fixes that can't change behavior; `--unsafe-fixes` needs you to review the diff.
- **PEP 723 scripts**: an inline `# /// script` header lists the dependencies, and `uv run script.py` builds a cached env for it. `uv lock --script` pins it. It's JBang for Python.
- **pytest discovery**: `test_*.py` files and `test_*` functions, with plain `assert` (rewritten to show both sides on failure).

Java vs Python

Concept	Java	Python	Watch out
Pick the runtime	`sdk use java 21-tem`	`uv python install 3.13` + `.python-version`	Libraries belong to one interpreter; a different `python3` sees none of them
New project	`mvn archetype:generate …`	`uv init --package --python 3.13 app`	Project `app-name`, import `app_name`
Build file	`pom.xml`	`pyproject.toml`	Ranges (`>=0.28.1`), not exact versions
Lockfile	Gradle lockfile (Maven: none)	`uv.lock`	Commit it; `uv sync` installs it exactly
Test scope	`<scope>test</scope>`	`uv add --dev pytest`	Lands in `[dependency-groups] dev`
Classpath	`java -cp target/classes:…`	`.venv` + `uv run …`	No `-cp` per run: bare `python3` = wrong env

Concept	Java	Python	Watch out
Run main	`mvn exec:java` / `Main-Class`	`uv run app-name` via `[project.scripts]`	Editable install: no rebuild after edits
Lint	Checkstyle, PMD, SpotBugs	`uv run ruff check`	Reports only; `--fix` to change files
Format	Spotless, google-java-format	`uv run ruff format`	Owns all whitespace; don't fight it
One-file program	JBang `//DEPS …`	PEP 723 header + `uv run script.py`	Unlocked scripts drift; `uv lock --script`
Shell	`jshell`	`uv run python` (`help`, `dir`, `type`)	Plain `python3` REPL can't see project packages
Unit test	JUnit `@Test` + `assertEquals`	`def test_x(): assert …`	Found by name, not annotation

Code snippets

New project, dependencies, run (uv 0.12.24)

```
uv init --package --python 3.13 log-insights   # pyproject, .python-version, src/log_insights/
cd log-insights
uv add httpx                     # runtime dependency (range) -> uv.lock updated
uv add --dev pytest ruff         # dev group, like test scope
uv run log-insights              # [project.scripts] entry point, inside .venv
uv lock --check                  # CI / pre-submit: fail if uv.lock is stale
uv lock --upgrade-package httpx  # deliberate upgrade of one package
```

Course ruff config (every task)

```
[tool.ruff]
line-length = 100

[tool.ruff.lint]
select = ["E", "F", "W", "I", "N", "B", "UP", "SIM", "RUF"]   # replaces the defaults
```

PEP 723 script

```
# /// script
# requires-python = ">=3.13"
# dependencies = ["httpx>=0.28.1"]
# ///
import httpx

print(httpx.get("https://httpbin.org/status/418").status_code)
```

First pytest test (pytest 9.1.1)

```
# tests/test_stats.py; run with: uv run pytest -q
from hello_uv.stats import mean


def test_mean_of_four():
    assert mean([1, 2, 3, 4]) == 2.5
```

Common pitfalls

1. **Mixing up uv and ruff**: "uv lints" or "`uv lock` is like Checkstyle" → both tools come from Astral → uv = runtimes, dependencies, environments; ruff = reads your code. (recap quizzes, week 1)

2. **Expecting a missing lock to install the lowest allowed version**: ranges resolve to the **newest** allowed, transitive dependencies included → a teammate months later gets different versions → commit `uv.lock`. (d2 check, recap quiz, week 1)
3. **Expecting `uv sync` to upgrade past the lock**: with a committed `uv.lock`, sync installs exactly the locked versions → upgrades happen only with `uv lock --upgrade[-package]` or a `pyproject.toml` change the lock no longer satisfies. (recap quiz, week 1)
4. **Matching `PATH` by directory instead of by name**: `python3` runs the first file named exactly `python3`; a folder holding only `python3.13` is skipped for `python3`. (recap quizzes, week 1)
5. **`uv init` without `--python`**: uv picks the first interpreter it finds (Homebrew 3.14 here) and writes `requires-python = ">=3.14"` → pass `--python 3.13`. (recap quiz, week 1)
6. **Assuming `select` adds rules, or that `check` fixes**: `select = ["I"]` turns F401 off entirely; `ruff check` only reports; formatting is `ruff format`'s job. (recap quiz, week 1)
7. **Running bare `python3 main.py`**: wrong interpreter, no project packages → `ModuleNotFoundError` → `uv run`.
8. **`pip install` into Homebrew's Python, or committing `.venv`**: pollutes the global env, or commits machine-specific junk → `uv add`; `.venv` stays gitignored.
9. **VS Code in Restricted Mode or on the wrong interpreter**: red squiggles on installed packages → trust the folder, select `./.venv/bin/python`.
10. **Running `--unsafe-fixes` blindly**: e.g. `!= None` → `is not None` can change behavior with a custom `__eq__` → read the diff.

Review next

- The uv ↔ Maven/SDKMAN and ruff ↔ Checkstyle/Spotless map, line by line.
- What `uv sync` does with and without a committed `uv.lock`, and which command upgrades on purpose.
- How the shell picks `python3` vs `python3.13` from `PATH`, and what `uv init` writes without `--python`.
- What `ruff check`, `ruff check --fix` and `ruff format` each do, and `select` vs `extend-select`.

Cheat sheet

Pythons

- `uv python list --only-installed`: what's installed
- `uv python install 3.13`: like `sdk install java`
- `.python-version`: per-project pin
- `which -a python3`: every `python3` on PATH, first wins
- PATH matches the exact name: `python3` ≠ `python3.13`

Projects

- `uv init --package --python 3.13 name`: new project, src layout
- `uv add pkg`: runtime dep (range) + lock
- `uv add --dev pytest ruff`: dev group (test scope)
- `uv remove pkg`
- `uv sync`: make `.venv` match `uv.lock` exactly
- `uv lock`: resolve ranges, write `uv.lock`
- `uv lock --check`: fail if lock is stale (CI)
- `uv lock --upgrade-package pkg`: deliberate bump
- No lock: newest allowed. Lock: exactly locked
- Commit `uv.lock`; never commit `.venv`

Running

- `uv run name`: `[project.scripts]` entry point
- `uv run python -m pkg`: runs `pkg/__main__.py`
- `uv run pytest -q` / `uv run ruff check`
- `uv run python`: REPL with project packages
- `uv run python -i file.py`: run, then REPL
- Bare `python3`: global env, `ModuleNotFoundError`

REPL

- `type(x)`, `dir(x)`, `help(x)` (`q` quits pager)
- `module.__file__`: where an import came from
- `exit` leaves the REPL

ruff

- `ruff check`: report findings (lint)
- `ruff check --fix`: apply safe fixes only
- `ruff check --statistics`: counts per rule
- `ruff check --show-settings`: active config
- `ruff format`: whitespace and wrapping
- `select` replaces defaults; `extend-select` adds
- Course: `E F W I N B UP SIM RUF`, line 100
- `# noqa: F841` silences one rule on one line

PEP 723 scripts

- `uv init --script s.py --python 3.13`
- `uv add --script s.py httpx`: edits header
- `uv lock --script s.py`: writes `s.py.lock`
- `uv run s.py`: cached env from the header

pytest

- Files `test_*.py`, functions `test_*`
- Plain `assert a == b`; failure shows both sides
- `.` per pass; exception in a test = failure

VS Code

- Trust the folder (Restricted Mode = no Pylance)
- Python: Select Interpreter → `./.venv/bin/python`
- Ruff extension: format and fix on save

Java → Python

- SDKMAN → uv python · Maven → uv · pom → pyproject
- Gradle lockfile → `uv.lock` · test scope → `--dev`
- classpath → `.venv` / `sys.path` · jshell → REPL
- Checkstyle → `ruff check` · Spotless → `ruff format`
- JBang → PEP 723 · JUnit → pytest

Official docs

- uv: working on projects: init, add, lock, sync, run (verified 2026-10-11)
- uv: managing dependencies: ranges, dependency groups (verified 2026-10-11)
- uv: running scripts: PEP 723 and `--script` commands (verified 2026-10-14)
- PEP 723: inline script metadata: the header format (verified 2026-10-14)
- Ruff: configuring Ruff and rules: `select`, rule codes (verified 2026-10-14)
- Ruff VS Code extension: format and fix on save (verified 2026-10-14)
- pytest: get started: discovery, `assert` (verified 2026-10-14)
- Python Packaging User Guide: writing pyproject.toml: `[project]` fields (verified 2026-10-11)
- VS Code: Python environments: interpreter selection (verified 2026-10-14)